# Unit Testing and Integration Testing in ASP.NET Core

## Practical testing strategies with MVC, EF Core, Docker, and Playwright

Boris Milašinović
University of Zagreb Faculty of Electrical Engineering and Computing
boris.milasinovic@fer.unizg.hr

31.3.2026.

Erasmus+ KA131 STA @ Faculty of Computer Science and Engineering,
Ss. Cyril and Methodius University of Skopje

# About the example materials

- The examples include data generation scripts, Docker container definitions, and the sample ASP.NET Core MVC application with appropriate tests
  - Main screens: Countries, People, Sports, Events, Registrations
  - Tests are split into three types: `UnitTests`, `IntegrationTests`, `UITests`
- The goal is to get familiar with what can be tested, how, and which tools can be used, not to go into the details of each individual test
  - The examples combine earlier code from lectures, improved and extended with assistance from Codex GPT-5.4, which was used to generate an extensive set of tests and to refine the use of the htmx library in the MVC application.
- The examples are available at

  `https://github.com/boris612/erasmus-sta-2026`

# Practical aspect of testing

- Shorten feedback cycles and make refactoring safer
  - Regression testing (re-running tests) ensures that previously tested features still work after changes in other parts of the system
- Serve as a form of documentation for system behavior
- Some parts of the system require a lot of setup to test manually, or simply take too much time; automation shortens the process
- Different test layers answer different questions:
  - Unit tests check isolated decision logic
  - Integration tests verify framework wiring and persistence
  - UI tests protect key user flows
- Focus on important and complicated parts where errors are more likely, instead of spending effort on trivial tests
  - Code coverage can be misleading. The goal is not the maximum number of tests, but useful confidence at a reasonable cost

# Few introductory notes

## Adam Freeman

"Unit testing is a tool and not a religion and only you know how much testing you require"

*Pro ASP.NET Core MVC, Sixth Edition, Apress*

## Edsger W. Dijkstra

"Testing can only show the presence of bugs, not their absence"

# Few introductory notes

## Adam Freeman

"Unit testing is a tool and not a religion and only you know how much testing you require"

*Pro ASP.NET Core MVC, Sixth Edition, Apress*

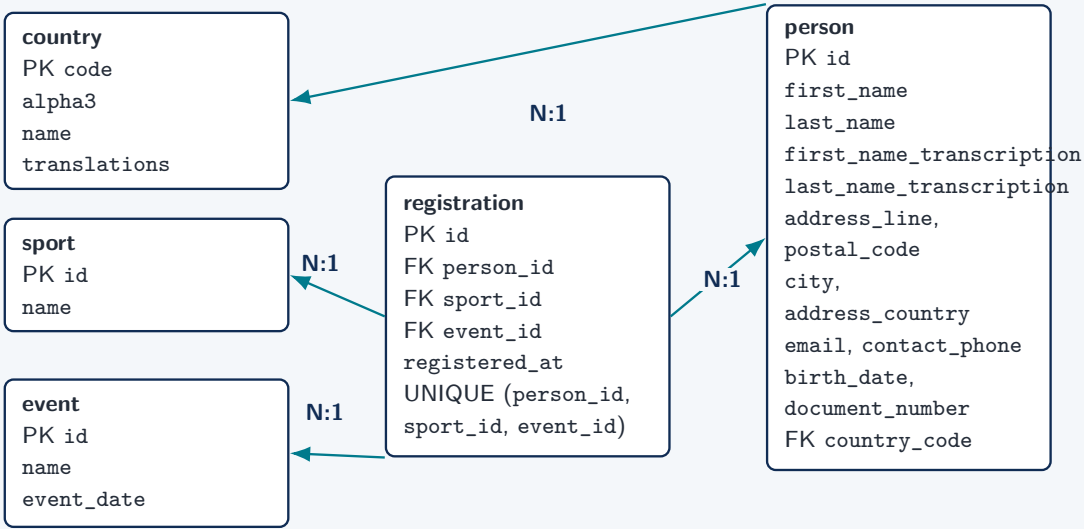
## Edsger W. Dijkstra

"Testing can only show the presence of bugs, not their absence"

## Disclaimer (Unofficial)

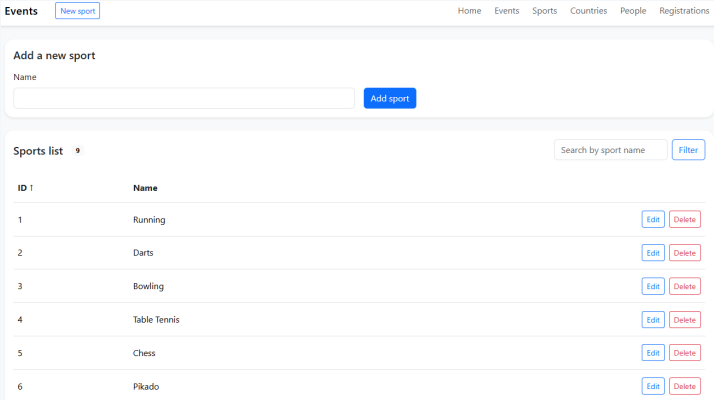
Use at your own risk. Testing reduces symptoms but does not guarantee a complete cure. Side effects may include failing tests, broken builds, and late-night debugging. If problems persist, consider stronger testing strategies.

# ER diagram of the example model



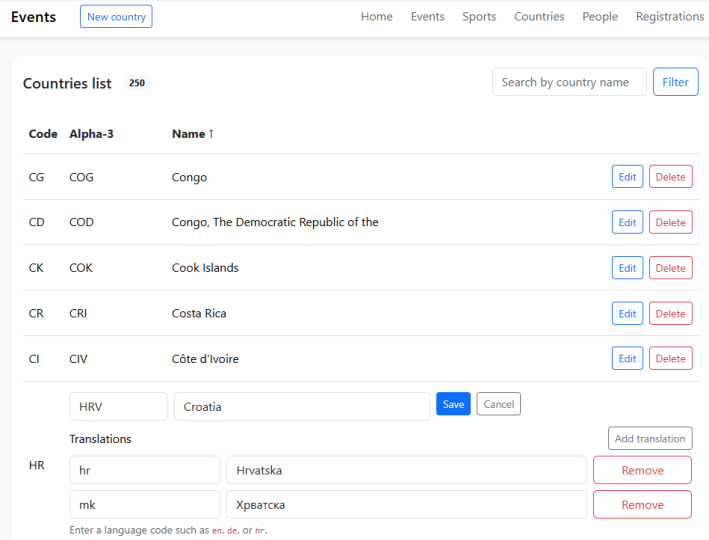
# Data preparation - manually writing simple scripts

- An example is the list of sports that can be created manually



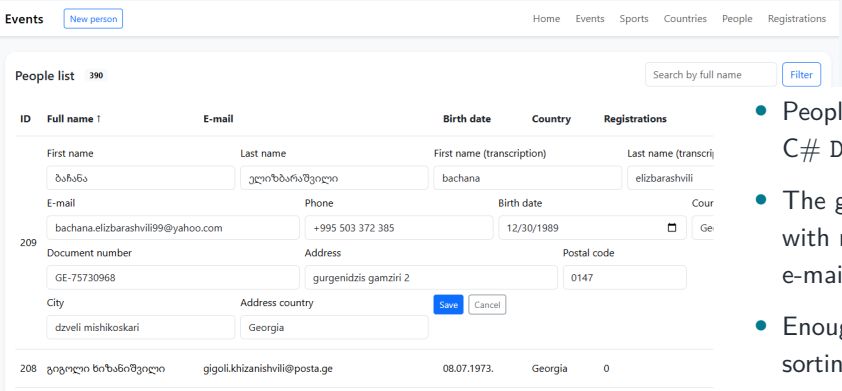

```
INSERT INTO sport (name) VALUES
('Running'),
('Darts'),
('Bowling'),
('Table Tennis'),
('Chess'),
('Pikado'),
('Mini Golf'),
('Swimming'),
('Cycling');
```

# Data preparation - use available libraries




- Countries are generated with a Python library `pycountry` that provides country names and translations in multiple languages
- Script used: `DataGenerator/create_countries_sql.py`

# Data preparation - create realistic datasets




- People were generated with the C# `DataGenerator` application
- The goal is a realistic dataset with names, addresses, and e-mails that look plausible
- Enough variety helps us test sorting, filtering, validation, and overall usability
- Different scripts and alphabets are especially useful because they immediately reveal whether the application handles them

# Generating realistic data with Bogus

- The generator uses the NuGet package `Bogus` with locale-specific fakers to produce more realistic names, addresses, cities, etc.
  - better demonstration datasets
  - repeatable test scenarios
  - checking whether the application handles different scripts correctly

```
foreach (var (locale, countryCode) in localeToCountryCode) {
    var faker = new Faker(locale);
    var firstName = faker.Name.FirstName();
    var lastName = faker.Name.LastName();
    var city = faker.Address.City();
    var contactPhone = faker.Phone.PhoneNumber();
    var birthDate = faker.Date.BetweenDateOnly(
        new DateOnly(1950, 1, 1), new DateOnly(2010, 12, 31));
```

# Preparing the demo database with Docker

- Prepare the database for demonstrations quickly and repeatably
- `docker-definitions/postgres-eventsdb/docker-compose.yml` defines PostgreSQL and pgAdmin
- The `init` folder contains:
  - SQL scripts for database schema and seed data
  - shell scripts (for users and permissions) that use values from `.env`
  - This makes it easy to change the database name, user, or password without rewriting the init scripts

**Commands to build and run, and delete containers and its volumes**

```
docker-compose up
docker-compose down -v
```

# Testing frameworks and other tools for .NET

- `xUnit` is used in all `Topic1` test projects
  - NUnit, and MSTest are also supported in Test Explorer in Visual Studio
  - The core idea is the same: *Arrange - Act - Assert*
    - `https://xunit.net/docs/comparisons`
    - `https://www.lambdatest.com/blog/nunit-vs-xunit-vs-mstest`
    - `https://docs.microsoft.com/en-us/dotnet/core/testing/#testing-tools`
- `Moq` is used where mocking is more convenient than creating a lightweight fake
- `Microsoft.EntityFrameworkCore.InMemory` helps create fast isolated test databases
- `Microsoft.AspNetCore.Mvc.Testing` provides `WebApplicationFactory` for creating a test web server
- `Playwright` is used in the UI test layer for browser-driven checks

# Running tests in Visual Studio

Test Explorer

70 65 2 3

Test run finished: 67 Tests (65 Passed, 2 Failed, 0 Skipped) run in 2,2 sec

| Test | Error Message | Duration | Traits |
|---|---|---|---|
| Events.Tests.IntegrationTests (27) | | 18,9 sec | |
| Events.Tests.IntegrationTests (27) | | 18,9 sec | |
| CountriesCrudShould (3) | | 1,8 sec | |
| EventsCrudShould (3) | | 1,9 sec | |
| EventsPageShould (1) | | 1,2 sec | |
| HomePageShould (1) | | 1,1 sec | |
| PeopleCrudShould (3) | | 2 sec | |
| PeopleGuardShould (1) | | 1,1 sec | |
| PeoplePageShould (1) | | 1,3 sec | |
| RegistrationsCrudShould (3) | | 2,1 sec | |
| RegistrationsGuardShould (2) | | 1,3 sec | |
| RegistrationsPageShould (3) | | 1,9 sec | |
| SportsCrudShould (4) | | 2 sec | |
| SportsPageShould (2) | | 1,4 sec | |
| Events.Tests.UITests (3) | | | |
| Events.Tests.UnitTests (40) | | 4,1 sec | |
| Events.Tests.UnitTests.Controllers (30) | | 3,2 sec | |
| CountriesControllerShould (5) | | 622 ms | |
| EventsControllerShould (5) | | 623 ms | |
| PeopleControllerShould (6) | | 674 ms | |
| CreatePerson | | 22 ms | |
| DeletePerson | | 1 ms | |
| EditPerson | | 558 ms | |
| RedirectToCountriesAndSetToastWhenIndexIsRequestedWithoutCountr... | | < 1 ms | |
| RedirectToCountriesAndSetToastWhenIndexIsRequestedWithoutCountr... | | 84 ms | |
| ReturnConflictWhenDeletingPersonWithRegistrations | | 9 ms | |
| RegistrationsControllerShould (7) | | 648 ms | |
| SportsControllerShould (7) | | 610 ms | |
| CreateSport | | 42 ms | |
| DeleteSport | | 2 ms | |
| EditSport | | 527 ms | |
| PopulateModelStateValidationErrorsForMissingName | This assertion intentio... | 1 ms | |

Run Debug

**Group Summary**

Tests in group: 67

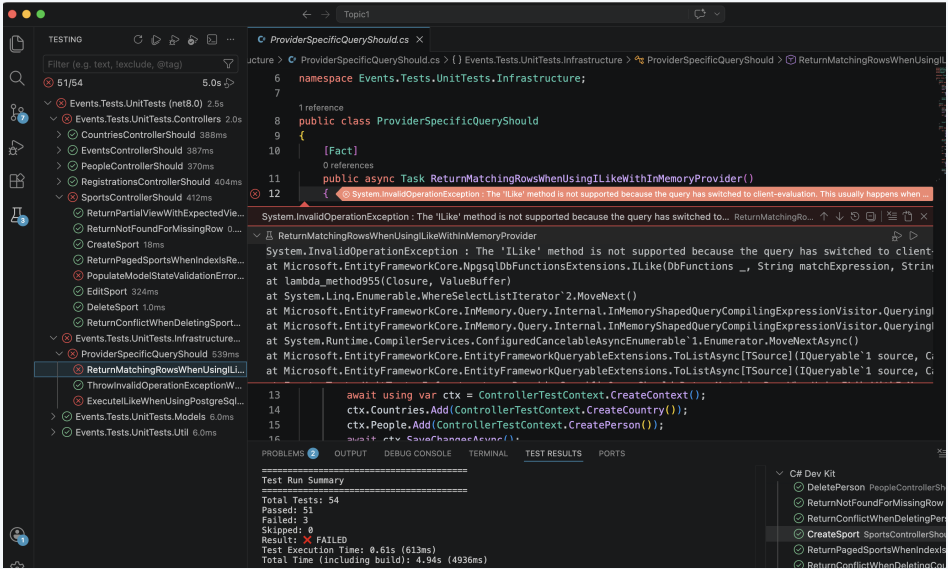
Total Duration: 23,1 sec

Outcomes

65 Passed

2 Failed

# Running tests in Visual Studio



# Testing types used in the examples: unit testing

- A unit test checks an individual part of the code
  - in object-oriented code, this is often a class and its behavior
- Unit tests run directly against controller actions and helper methods
- What can be tested for controllers?
  - route targets in redirects
  - returned result type or partial view
  - mapped view model values
  - status-dependent response headers such as `HX-Trigger`
- Only code within the developer's control is tested, without involving infrastructure concerns
  - An isolated `EventsContext` with EF Core In-Memory
  - Dependencies such as paging options, `Sieve`, or `TempData` are supplied explicitly

# Characteristics of (not only) unit tests

- **Fast**: in milliseconds, e.g., projects can have thousands of unit tests.
- **Isolated**: unit tests should be standalone and run in isolation, without dependencies on outside factors, or order of tests.
- **Repeatable**: tests can be repeated and must be consistent, i.e., yield the same results if inputs are not changed.
- **Self-checking**: test result should be determined without human interaction.
- **Timely**: time for writing a test should not be disproportionately longer compared to writing the code being tested.

# Testing types used in the examples: integration testing

- Integration testing ensures that multiple components function correctly together with supporting infrastructure such as the database, web server, and network
- Typical checks include:
  - redirect behavior for guard scenarios
  - CRUD happy paths for MVC screens
  - filtering, paging, and HTMX partial responses
- Integration tests exercise the application through HTTP using `WebApplicationFactory` that creates a test web server
- They verify routing, filters, model binding, Razor views, and EF Core together
- `Topic1` uses a custom factory that swaps the app database with an isolated in-memory test database

# Testing types used in the examples: UI tests

- UI tests exercise the application through the browser from the user's point of view
- In `Topic1`, they help us verify that the rendered pages and core user flows still work end-to-end
- Typical checks include:
  - opening the home page and navigating to main screens
  - interacting with the page through the actual UI
  - confirming that the visible behavior matches expected results after user actions
- These tests are slower than unit and integration tests, but they give strong confidence in the full stack

# Different approaches and recommendations for test naming

- https://dzone.com/articles/7-popular-unit-test-naming
  - MethodName_StateUnderTest_ExpectedBehavior
    - MethodName_Scenario_ExpectedBehavior (https://docs.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices)
  - MethodName_ExpectedBehavior_StateUnderTest
  - test[Feature being tested]
  - Feature to be tested
  - Should_ExpectedBehavior_When_StateUnderTest
  - When_StateUnderTest_Expect_ExpectedBehavior
  - Given_Preconditions_When_StateUnderTest_Then_ExpectedBehavior
  - test[FeatureBeingTested]
- Choose one convention per project or team and apply it consistently
- Test class names and namespaces should also make the test scope obvious
- Long names are acceptable when they remove ambiguity

# Test names in the example

- Guidance on naming test classes and organizing namespaces
  - `https://ardalis.com/unit-test-naming-convention`
- The repository favors descriptive names that read like specifications
- The test class usually describes the subject:
  - `PeopleControllerShould`
  - `RegistrationsGuardShould`
  - `PagingInfoTests`
- The test method describes the expected behavior:
  - `RedirectToCountriesAndSetToastWhenIndexIsRequestedWithoutCountries`
  - `ReturnConflictWhenDeletingPersonWithRegistrations`

# A unit test example

```
[Fact]
public void TotalPages_RoundsUpWhenFilteredItemsDoNotDivideEvenly() {
    var pagingInfo = new PagingInfo {
        FilteredItemsCount = 21,
        ItemsPerPage = 10,
        CurrentPage = 1
    };
    Assert.Equal(3, pagingInfo.TotalPages);
}
```

- We arrange the object under test, execute the logic, and verify the expected result
- `[Fact]` marks a parameterless test case in xUnit
  - Use `[Trait("Category", "Unit")]` for grouping and filtering tests
- Besides `Assert.Equal`, common assertions include `True`, `False`, `Null`, `NotNull`, `Contains`, `Empty`, and `Throws`

# A theory test with InlineData

```
[Theory]
[InlineData(1, 10, 1)]      [InlineData(10, 10, 1)]
[InlineData(11, 10, 2)]     [InlineData(21, 10, 3)]
public void TotalPages_ReturnsExpectedPageCount(
    int filteredItemsCount, int itemsPerPage, int expectedTotalPages) {
    var pagingInfo = new PagingInfo {
        FilteredItemsCount = filteredItemsCount,
        ItemsPerPage = itemsPerPage
    };
    Assert.Equal(expectedTotalPages, pagingInfo.TotalPages);
}
```

- `[Theory]` lets us run the same test logic with multiple input combinations
- `[InlineData]` initializes arguments; the values must be compile-time constants
- This is useful when the same behavior should hold for several representative cases

# A theory test with MemberData

```
public static IEnumerable<object[]> TotalPagesCases => [
                        [5, 10, 1],     [20, 10, 2],    [25, 10, 3]  ];
[Theory] [MemberData(nameof(TotalPagesCases))]
public void TotalPages_ReturnsExpectedPageCount_WhenUsingMemberData(
    int filteredItemsCount, int itemsPerPage, int expectedTotalPages) {
    var pagingInfo = new PagingInfo {
        FilteredItemsCount = filteredItemsCount,
        ItemsPerPage = itemsPerPage
    };
    Assert.Equal(expectedTotalPages, pagingInfo.TotalPages);
}
```

- `[MemberData]` loads test cases from a property, field, or method
- It is useful when the data set is larger, reused, or too complex for `[InlineData]`
- The test logic stays the same, but the data source becomes easier to maintain

# Database registration and constructor injection in the application

```
builder.Services.AddDbContext<EventsContext>(options =>
    options.UseNpgsql(
        builder.Configuration.GetConnectionString("EventDB")));

public class SportsController : Controller
{
    public SportsController(
        EventsContext ctx,
        ISieveProcessor sieveProcessor,
        IOptions<PagingSettings> pagingSettings)
    {
        ...
    }
}
```

# Testing with an in-memory database

```
public static EventsContext CreateContext()
{
    var options = new DbContextOptionsBuilder<EventsContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;

    return new EventsContext(options);
}
```

- Database context is created and passed manually to a controller
- Each unit test gets an isolated database instance
- Seed only the entities needed for the behavior under test

# Seeding initial data to the database

```
var options = new DbContextOptionsBuilder<EventsContext>()
                    .UseInMemoryDatabase(Guid.NewGuid().ToString())
                    .Options;
await using var ctx = new EventsContext(options);
ctx.Sports.AddRange(
    new Sport { Id = 1, Name = "Athletics" },
    new Sport { Id = 2, Name = "Basketball" },
    new Sport { Id = 3, Name = "Cycling" });
await ctx.SaveChangesAsync();
```

- Before a test, we often want a few known records already in the database
- This kind of seed method keeps the setup explicit and easy to reuse

# Solving dependency injection in controllers

```
private static SportsController CreateController(EventsContext ctx, bool
    useSieve = true) {
    return new SportsController(
        ctx,
        useSieve ? ControllerTestContext.CreateSieveProcessor() : null!,
        ControllerTestContext.CreatePagingOptions())
        .WithTempData();
}
```

- It creates the controller with test-ready dependencies instead of relying on the full ASP.NET Core runtime
- The helper keeps individual tests shorter and makes the setup consistent across multiple test methods

# Controllers that use TempData

```
public static T WithTempData<T>(this T controller) where T : Controller
{
    var httpContext = new DefaultHttpContext();
    controller.ControllerContext = new ControllerContext
    {
        HttpContext = httpContext
    };
    controller.TempData = new TempDataDictionary(
        httpContext,
        new TestTempDataProvider());
    return controller;
}
```

- This is convenient when we want a lightweight working `TempData` implementation in tests

# Mocking TempData with Moq

```
var tempDataMock = new Mock<ITempDataDictionary>();
var controller = new PeopleController(ctx, null!, optionsMock.Object)
{
    TempData = tempDataMock.Object
};

var result = await controller.Index(new SieveModel());

tempDataMock.VerifySet(
    t => t["ToastMessage"] =
        "At least one country must be created before adding people.",
    Times.Once);
```

- A mock is useful when we want to verify that the controller interacted with `TempData`

# When to use real TempData vs. a mock

- Use `WithTempData()` when you want a simple working test double and readable assertions on stored values
- Use a mock when interaction itself matters and you want explicit verification with `Verify` or `VerifySet`
- A mock is also useful when an interface has a larger or more complicated implementation, but the test needs only a small part of that behavior
- We usually mock interfaces or overridable members on classes; plain non-virtual concrete class members cannot be mocked directly with tools such as Moq
- Both approaches are valid; the choice depends on what exactly the test is trying to prove

# Testing controller results

```
var result = await controller.Index(null, new SieveModel());
var redirect = Assert.IsType<RedirectToActionResult>(result);
Assert.Equal("Events", redirect.ControllerName);
```

- Here we check the controller result type and the exact redirect target
- In the `RegistrationsController` example, the action should redirect when no events exist (above) in contrast to returning partial view with data (below) in which we can verify the returned view type and and, and produced data

```
var result = await controller.Index(new SieveModel {
    Filters = "FirstName==Ana"
});
var partial = Assert.IsType<PartialViewResult>(result);
Assert.Equal("_PeopleList", partial.ViewName);
Assert.Contains(ctx.People, p => p.FirstName == "Ana");
```

# What these units test gives us

- Focus on controller decision logic only
- No HTTP server, routing, filters, or middleware
- Assertions are simple and precise:
  - action result type
  - redirect target
  - view model contents
  - TempData / validation behavior

# Note on EF Core and in-memory databases

- Why EF Core In-Memory is useful
  - Very fast setup for controller + HTTP + persistence scenarios
  - Great for smoke tests, redirects, guards, and CRUD happy paths
  - Easy seeding through custom factory hooks
- When EF Core or SQLite in-memory is not enough
  - It is not a full substitute for the production database provider
  - SQL translation differences may be hidden
  - Query translation, relational constraints, and transaction behavior can differ
  - For higher confidence, a real database may be needed
  - Docker makes this practical with disposable databases with initialization scripts

# Provider-specific queries and in-memory limits

```
var people = await ctx.People
    .Where(person => EF.Functions.ILike(
        person.FirstName, "%iv%"))
    .ToListAsync();
```

- The test `ReturnMatchingRowsWhenUsingILikeWithInMemoryProvider` highlights a provider-specific query
- `EF.Functions.ILike(...)` is a PostgreSQL feature and is not supported by EF Core In-Memory
- This is why some query behavior must be verified with the real database provider
- We solved this by creating a separate `docker-compose` setup on a different port.
  - See `ExecuteILikeWhenUsingPostgreSqlProvider` method

# Where unit tests stop

- They do not prove that the endpoint is reachable through HTTP
- They do not exercise model binding, filters, middleware, or view rendering
- They do not validate realistic application startup and dependency wiring
- This is why we add integration tests rather than replacing unit tests

# Validation does not run automatically in unit tests

```
var invalidModel = new SportViewModel { Name = string.Empty };
var result = await controller.Create(invalidModel,
    ControllerTestContext.EmptySieveModel());

Assert.False(controller.ModelState.IsValid);
Assert.True(controller.ModelState.ContainsKey(
    nameof(SportViewModel.Name)));
```

- The test `PopulateModelStateValidationErrorsForMissingName` demonstrates a naive expectation
- In plain unit tests, the MVC validation pipeline does not run automatically
- If we need real validation behavior, we must populate `ModelState` ourselves or test through the MVC pipeline

# Integration testing

- Integration tests answer the question whether the application behave correctly over HTTP
- The integration test starts a test web server and sends real HTTP requests to it
- They include middleware, routing, filters, views, model binding, and EF Core interaction
- This gives broader confidence than unit tests, while still running faster than browser automation

# How the custom test host is built (1)

```
public class CustomWebApplicationFactory : WebApplicationFactory<Program> {
  private readonly Action<EventsContext>? seed;
  private readonly InMemoryDatabaseRoot databaseRoot = new();
  private readonly string databaseName = Guid.NewGuid().ToString();
  public CustomWebApplicationFactory(Action<EventsContext>? seed = null) {
      this.seed = seed;
  }
  protected override void ConfigureWebHost(IWebHostBuilder builder) {
    services.RemoveAll(typeof(DbContextOptions<EventsContext>));
    services.RemoveAll(
      typeof(IDbContextOptionsConfiguration<EventsContext>));
    services.RemoveAll(typeof(EventsContext));
    services.AddDbContext<EventsContext>(options =>
        options.UseInMemoryDatabase(databaseName, databaseRoot));
    ...
```

# How the custom test host is built (2)

```
protected override void ConfigureWebHost(IWebHostBuilder builder) {
    builder.UseEnvironment("Development");
    ... replace services related to database context ...

    var serviceProvider = services.BuildServiceProvider();
    using var scope = serviceProvider.CreateScope();
    var ctx = scope.ServiceProvider.GetRequiredService<EventsContext>();
    ctx.Database.EnsureCreated();
    seed?.Invoke(ctx);
    ctx.SaveChanges();
  });
}
```

- The custom factory replaces the app database registration for test execution
- The same factory can also seed scenario-specific data before a request is sent

# Basic integration testing

```
[Fact]
public async Task
    RedirectToSportsWhenRegistrationsPageIsRequestedWithoutSports() {
    await using var factory = new CustomWebApplicationFactory();
    using var client = factory.CreateClient(
        new WebApplicationFactoryClientOptions {
            AllowAutoRedirect = false
        });

    var response = await client.GetAsync("/Registrations");

    Assert.Equal(HttpStatusCode.Redirect, response.StatusCode);
    Assert.Equal("/Sports", response.Headers.Location?.OriginalString);
}
```

# Basic integration testing with context seeding

```
[Fact]
public async Task
    RedirectToPeopleWhenRegistrationsPageIsRequestedWithoutPeople() {
    await using var factory = new CustomWebApplicationFactory(ctx => {
        ctx.Sports.Add(new Sport { Id = 10, Name = "Football" });
        ctx.Events.Add(new Event { Id = 100, Name = "Spring Games",
            EventDate = new DateOnly(2026, 4, 15) }); });
    using var client = factory.CreateClient(
        new WebApplicationFactoryClientOptions {
            AllowAutoRedirect = false
        });
    var response = await client.GetAsync("/Registrations");
    Assert.Equal(HttpStatusCode.Redirect, response.StatusCode);
    Assert.Equal("/People", response.Headers.Location?.OriginalString);
}
```

# Integration tests catch model validation errors

```
[Fact]
public async Task ReturnValidationErrorWhenCreatingSportWithoutName() {
    await using var factory = new CustomWebApplicationFactory();
    using var client = factory.CreateClient();
    var response = await AntiforgeryRequestHelper.PostFormAsync(
        client, "/Sports", "/Sports/Create",
        [new("Name", string.Empty)]);
    var html = await response.Content.ReadAsStringAsync();
    Assert.Contains("The Name field is required.", html);
}
```

- Here the request goes through the MVC pipeline, so model validation is actually executed. That is why integration testing catches the validation error that we could not rely on in a plain unit test

# Integration testing a create action

```
[Fact]
public async Task CreateSport() {
    await using var factory = new CustomWebApplicationFactory();
    using var client = factory.CreateClient();

    var response = await AntiforgeryRequestHelper.PostFormAsync(
        client, "/Sports", "/Sports/Create",
        [new("Name", "Volleyball")]);
    var html = await response.Content.ReadAsStringAsync();
    Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    Assert.Contains("Volleyball", html);
}
```

- This example shows the full POST request used to create a new sport through the MVC application verifying both the HTTP result and the returned HTML content

# Recommended integration test scenarios

- GET requests return expected status codes and views/partials
- Redirect guards work when prerequisite data is missing
- Create, edit, and delete flows succeed through HTTP
- Invalid input returns validation messages
- Database conflicts produce readable `ProblemDetails`
- Filtering, sorting, and paging behave correctly with seeded data

# UI testing with Playwright

- For browser tests, the application must be running
- Custom helper (see `UiTestHarness`) starts the MVC app, launch an instance of a browser using Playwright, and opens required page for further interaction

```
await using var harness = await UiTestHarness.CreateAsync();
await harness.Page.GotoAsync($"{harness.RootUrl}/Sports");
```

- Without modifications, the UI would connect to real application database
  - To avoid modifying `Program.cs` helper replace the connection string with the connection string of the test database (defined as `EventDB-Test` key)

# UI testing with Playwright: verifying the result

```
var expectedSports = new[] { "Running", "Chess", "Swimming" };

foreach (var expectedSport in expectedSports)
{
    await Assertions.Expect(harness.Page.Locator("#sports-list"))
        .ToContainTextAsync(expectedSport);
}

await Assertions.Expect(harness.Page.GetByRole(AriaRole.Heading,
    new() { Name = "Add a new sport" })).ToBeVisibleAsync();
}
```

# UI testing with Playwright - filling forms and submitting data

```
[Fact]
public async Task SportsPageShouldCreateSportAndShowSuccessToast() {
    await using var harness = await UiTestHarness.CreateAsync();

    await harness.Page.GotoAsync($"{harness.RootUrl}/Sports");
    await harness.Page.GetByRole(AriaRole.Button,
                                 new() { Name = "New sport" })
                      .ClickAsync();
    var createForm = harness.Page.Locator("#create-sport-form");
    await createForm.GetByLabel("Name", new() { Exact = true })
                    .FillAsync("Volleyball");
    await createForm.GetByRole(AriaRole.Button,
                               new(){ Name = "Add sport" })
                    .ClickAsync();
```

# What Playwright adds

- Verifies the application exactly as a user sees it
- Catches broken selectors, layout flow, JavaScript behavior, and browser integration
- Playwright can also record user interactions and generate starter test code from them
- Best used for:
  - smoke navigation
  - 1–2 critical CRUD journeys
  - regressions around dynamic UI behavior
- Keep the suite intentionally small to preserve maintainability

# Takeaways

- No single test type is enough on its own
  - Unit tests: fastest, narrowest feedback (for branches, guards, mappings)
  - Integration tests: verify framework wiring, middleware, routing, model binding, filters, and persistence
  - UI tests: highest confidence from the user perspective, but also the slowest
- Focus on important and complicated parts where errors are more likely, instead of spending effort on trivial tests
- Realistic data matters; tools such as Bogus make demos and tests much more convincing
  - EF Core In-Memory is a convenience, not a full database substitute
  - Docker and Playwright extend confidence into realistic scenarios
- Keep the overall strategy balanced: strong confidence, fast feedback, and a maintainable test suite